Tensor Networ methods workshop II
ED vs DMRG

November 4, 2026

Github with codes: `https://github.com/mkpanfil/TN_UW`

Source about `XDiag.jl`:`https://awietek.github.io/xdiag/user_guide/`

Exemplary IDE: `https://vscodium.com/`

Basic setup: Julia language, precompiling, multithreading

Please refer to the file
`https://github.com/mkpanfil/TN_UW/blob/main/julia_MKL_instructions.txt`

XDiag.jl working example

Check if you installed julia and XDiag.jl properly by running
`https://github.com/mkpanfil/TN_UW/blob/main/XDiag_free_fermions_example.jl`.

Itensors, ItensorsMPS

Itensors: `https://itensor.github.io/ITensors.jl/v0.1/`
ItensorMPS (DOCUMENTATION HERE): `https://docs.itensor.org/ITensorMPS/stable/`

1 Transverse Field Ising model, but DMRG now

The phase diagram of the TFI model looks like this:

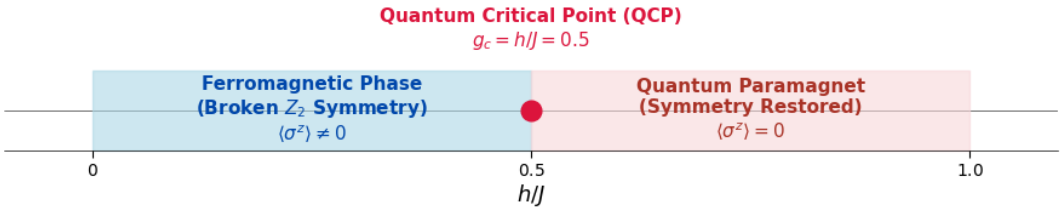


Figure 1: Transverse Field Ising phase diagram

The Hamiltonian of Transverse field Ising model is given by:

$$H_{\mathrm{TFI}} = -J \sum_{i=1}^{L-1} S_i^z S_{i+1}^z - h \sum_{i=1}^{L} S_i^x, \tag{1}$$

where J gives us strength of the interaction in the (z-direction) while h is the applied transverse field (x-direction) and S_i^x, S_i^z are spin-1/2 operators acting on site i, where we have sites going from site 1 up to L.

1.1 Exercise: Find the ground state (lowest lying energy) of TFI model via MPS DMRG

Simple example code for Ising without field

```
sites = siteinds("S=1/2", N)
os = OpSum()
for j in 1:(N - 1)
    os += -J, "Sz", j, "Sz", j + 1
end
H = MPO(os, sites)
psi0 = random_mps(sites)
energy, psi = dmrg(H, psi0; nsweeps, maxdim, cutoff)
```

Compute the entanglement entropy of the GS of TFI. Select small size like $L = 10$ and do comparison between entanglement entropy results of ED and DMRG in a plot.

2 XXZ model (fixed magnetisation in ED and DMRG)

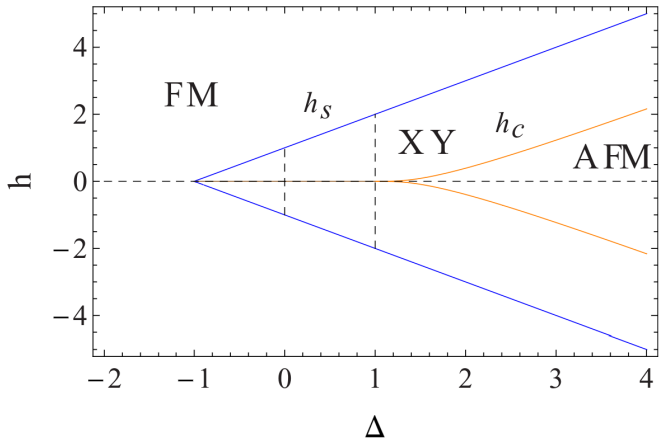


Figure 2: Parrallel field XXZ spin-1/2 model phase diagram
`https://journals.aps.org/prb/pdf/10.1103/PhysRevB.93.054417`

The Hamiltonian of the XXZ spin-1/2 model is given by:

$$H_{\mathrm{XXZ}} = J(\sum_{i=1}^{L-1} S_i^x S_{i+1}^x + S_i^y S_{i+1}^y + \Delta S_i^z S_{i+1}^z) - h\sum_{i=1}^{L} S_i^z, \tag{2}$$

or alternatively:

$$H_{\mathrm{XXZ}} = J(\sum_{i=1}^{L-1} S_i^+ S_{i+1}^- + S_i^- S_{i+1}^+ + \Delta S_i^z S_{i+1}^z) - h\sum_{i=1}^{L} S_i^z, \tag{3}$$

In XDiag.jl in ED instead of applying field h^z do the fixed magnetizations for a given value of Δ (for simplicity $\Delta = 1$). By taking differences of energies at fixed magnetizations M like:

$$h_{M,M-1} = E(M) - E(M-1) \tag{4}$$

To set fixed magnetisation on the level of a block you can do:

```
block = Spinhalf(N, nup)
```

where `nup` is number of spins up.

Do the plot of magnetisations as a function of a field:
a) where you have Hamiltonian with a field explicitly
b) where you have Hamiltonian without a field but with a fixed magnetisation. Both in the case of XDiag.jl (ED) and in the ItensorMPS.jl (DMRG) version.

To set the magnetisation in DMRG you set quantum numbers conservation and prepare initial state with selected number of spins up:

```
sites = siteinds("S=1/2", N; conserve_sz=true)

state = Vector{String}(undef, N)
#(here fill state with selected number of "Up" and "Down" to much magnetisation)
MPS(sites, state)
```

3 Symmetries, exercise: ED only now using symmetry settings in XDiag.jl to measure momentum values

In XDiag.jl you can apply symmetries: `https://awietek.github.io/xdiag/user_guide/11-symmetries/`.
Below for simplicity, how to apply symmetries:

```
group_elements = [Permutation([mod1(i + r, N) for i in 1:N]) for r in 0:(N - 1)]
group = PermutationGroup(group_elements)
 k = 2 * pi * q / N
 characters = [exp(-1im * k * r) for r in 0:(N - 1)]
irrep = Representation(group, characters)
block = Spinhalf(N, irrep) #such block defines selected momentum region
```

Do plots as a function of momentum k for every value of h, just like entanglement entropy plots in previous exercise.

This only works in the cases where singlet-singlet gap is exactly the same as singlet-triplet gap. This is not true for example for general J_2 value in $J_1 - J_2$ model.

Author: *Jędrzej Wardyn*